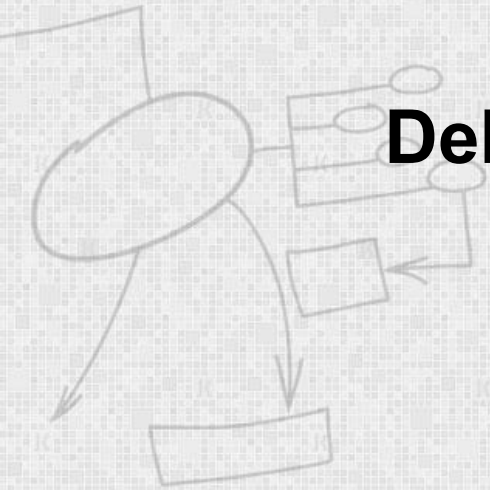
EC-360
SOFTWARE ENGINEERING

Lecture 12

Debugging & Integration Testing

By
Dr Wasi Haider Butt



Testing and debugging

Testing and **debugging** are distinct processes

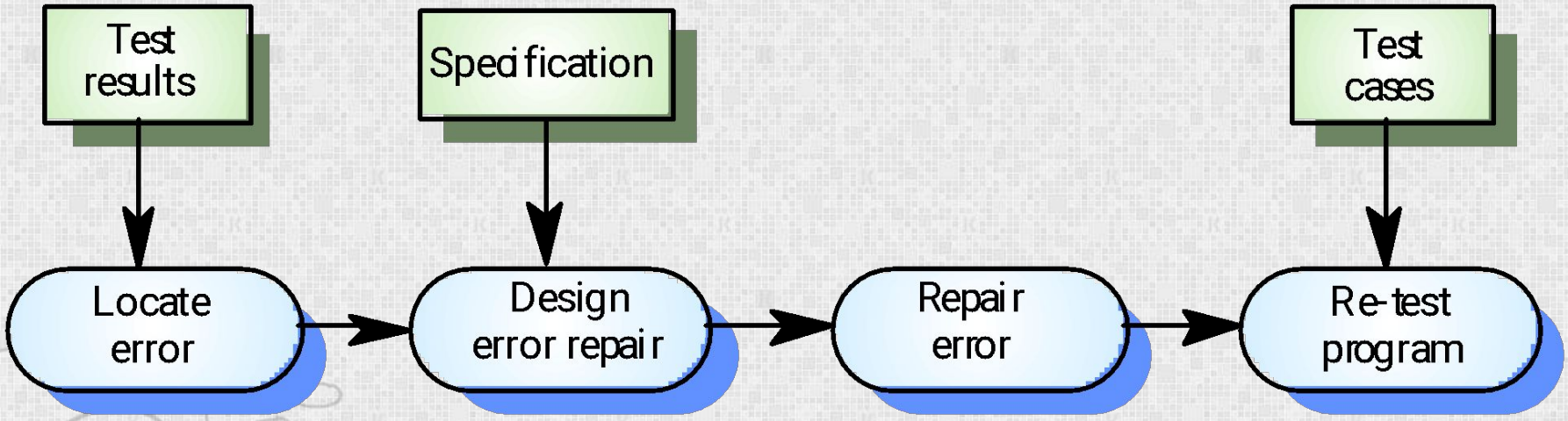
- Testing is concerned with establishing the existence of defects in a program

Debugging is concerned with

- locating and

- repairing these errors

The debugging process



Debugging approaches

- **Brute Force Method:**
- This is the least efficient method.
- In this approach, the program is loaded with print statements to print the intermediate values with the hope that some of the printed values will help to identify the statement in error.

Debugging approaches

- **Backtracking:**
- This is also a fairly common approach.
- In this approach, beginning from the statement at which an error symptom has been observed, the source code is traced backwards until the error is discovered.
- As the number of source lines to be traced back increases, the number of potential backward paths increases and may become unmanageably large thus limiting the use of this approach.

Debugging approaches

- **Program Slicing:**
- Here the search space is reduced by defining slices.
- A slice of a program for a particular variable at a particular statement is the set of source lines preceding this statement that can influence the value of that variable

```
int i;
int sum = 0;
int product = 1;
int w = 7;
for(i = 1; i < N; ++i) {
  sum = sum + i + w;
  product = product * i;
}
write(sum);
write(product);
```

```
int i;
int sum = 0;
int w = 7;
for(i = 1; i < N; ++i) {
  sum = sum + i + w;

}
write(sum);
```

Program analysis tools

- Automated tool that takes the source code or the executable code of a program as input and produces reports regarding several important characteristics of the program, such as its size, complexity, adequacy of commenting, adherence to programming standards, etc.

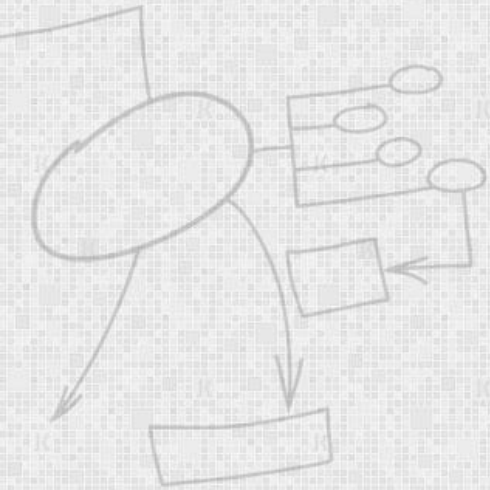
- Two broad categories of program analysis tools:
  - Static Analysis tools
  - Dynamic Analysis tools

Static program analysis tools

- It assesses and computes various characteristics of a software product without executing it.
- Typically, static analysis tools analyse some structural representation of a program to arrive at certain analytical conclusions, e.g. that some structural properties hold.

Static program analysis tools

- Whether the coding standards have been adhered to?
- Code walk through might be considered as static analysis methods.



Dynamic program analysis tools

- Dynamic program analysis techniques require the program to be executed and its actual behaviour recorded.
- A dynamic analyser usually instruments the code (i.e. adds additional statements in the source code to collect program execution traces).
- The instrumented code when executed allows us to record the behaviour of the software for different scenarios

Dynamic program analysis tools may reveal

- **Resources consumed** - the time of program execution on the whole or its modules individually, the number of external queries (for example, to the database), the number of memory being used, and other resources;
- **Cyclomatic complexity**, the degree of code coverage with tests, and other program metrics;
- **Program errors** - division by zero, null pointer dereferencing, memory leaks,
- **Vulnerabilities in the program**

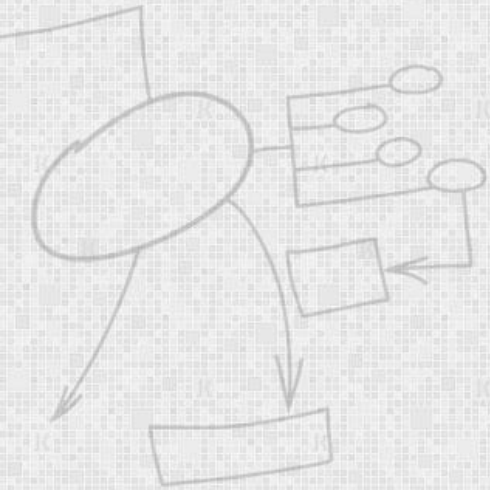
Integration testing

- The primary objective of integration testing is to test the module interfaces, i.e. there are no errors in the parameter passing, when one module invokes another module.

- During integration testing, different modules of a system are integrated in a planned manner using an integration plan.

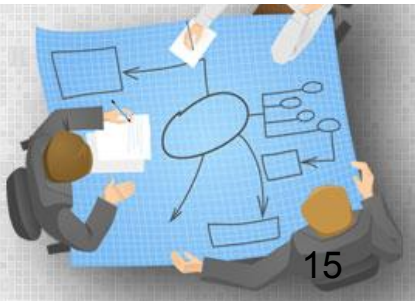
Integration test approaches

- Big bang approach
- Top-down approach
- Bottom-up approach
- Mixed-approach



Terminology

- **Stub** – the dummy modules that simulates the low level modules.
- **Driver** – the dummy modules that simulate the high level modules.



What Is Big Bang Testing?

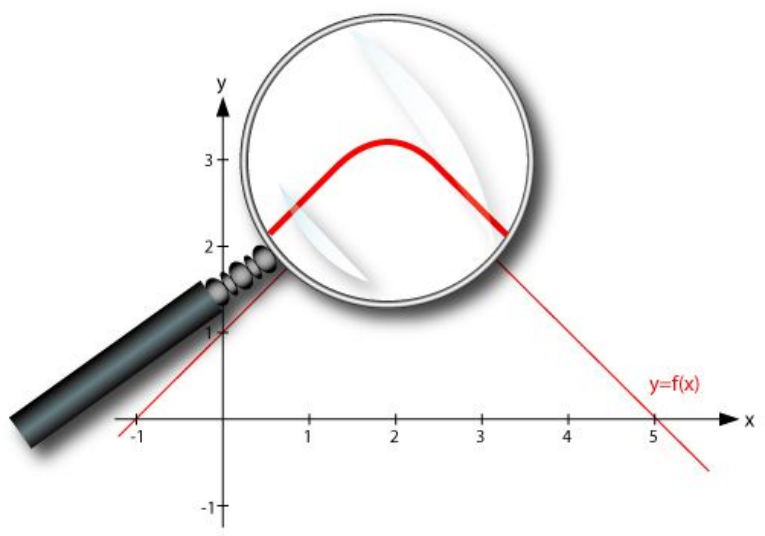
- In Big Bang Integration testing, individual modules of the programs are not integrated until **every thing** is ready. It is called 'Run it and see' approach.
- In this approach, the program is integrated **without any formal integration testing**, and then run to ensures that all the components are working properly.

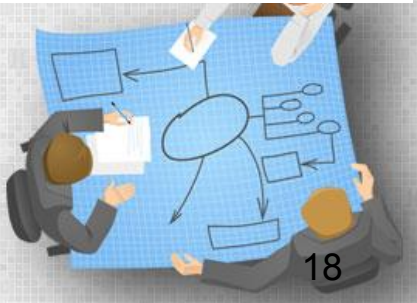
Disadvantages of Big Bang

- Defects present at the interfaces of components are identified at **very late stage**.
- It is very difficult to isolate the defects found, as it is very difficult to tell **whether defect is in component or interface**.
- There is **high probability of missing** some critical **defects** which might surfaced in production.
- It is very difficult to make sure that all the cases for integration testing are covered.

Big Bang Testing: Conclusion

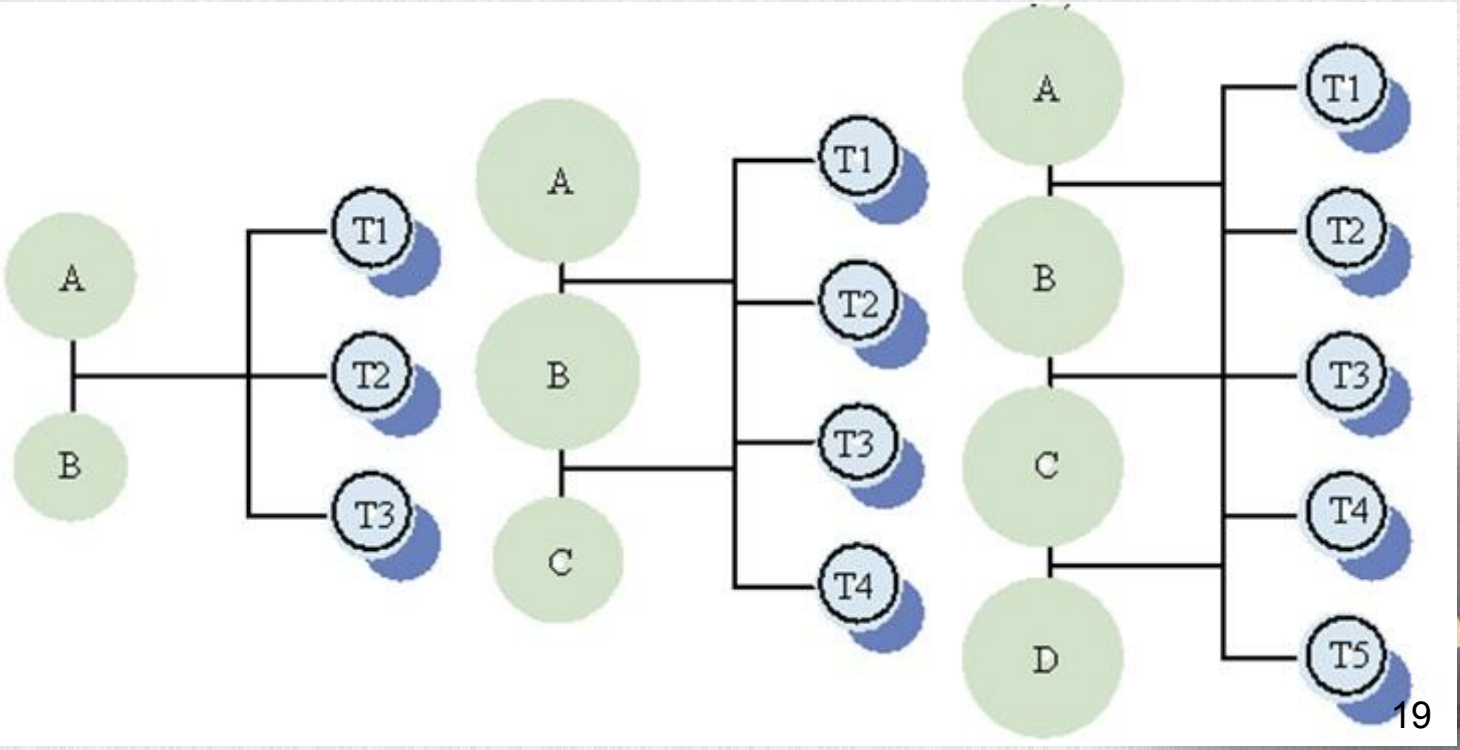
- This is **not the way** you should integrate and test software!
- But it might be good with this assumption applied:
  - for small systems, **but not for enterprise level applications**.





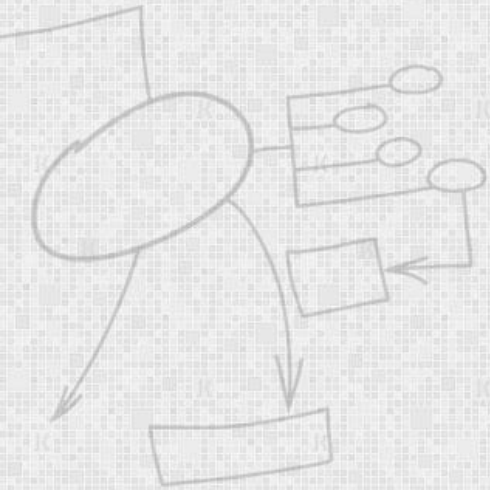
How To Integrate?

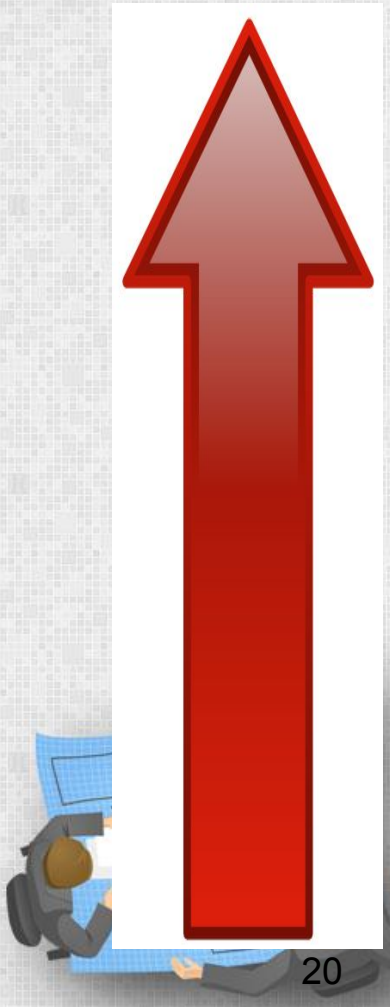
- If Big Bang integration is bad, then how to integrate?
- The answer: incremental integration.



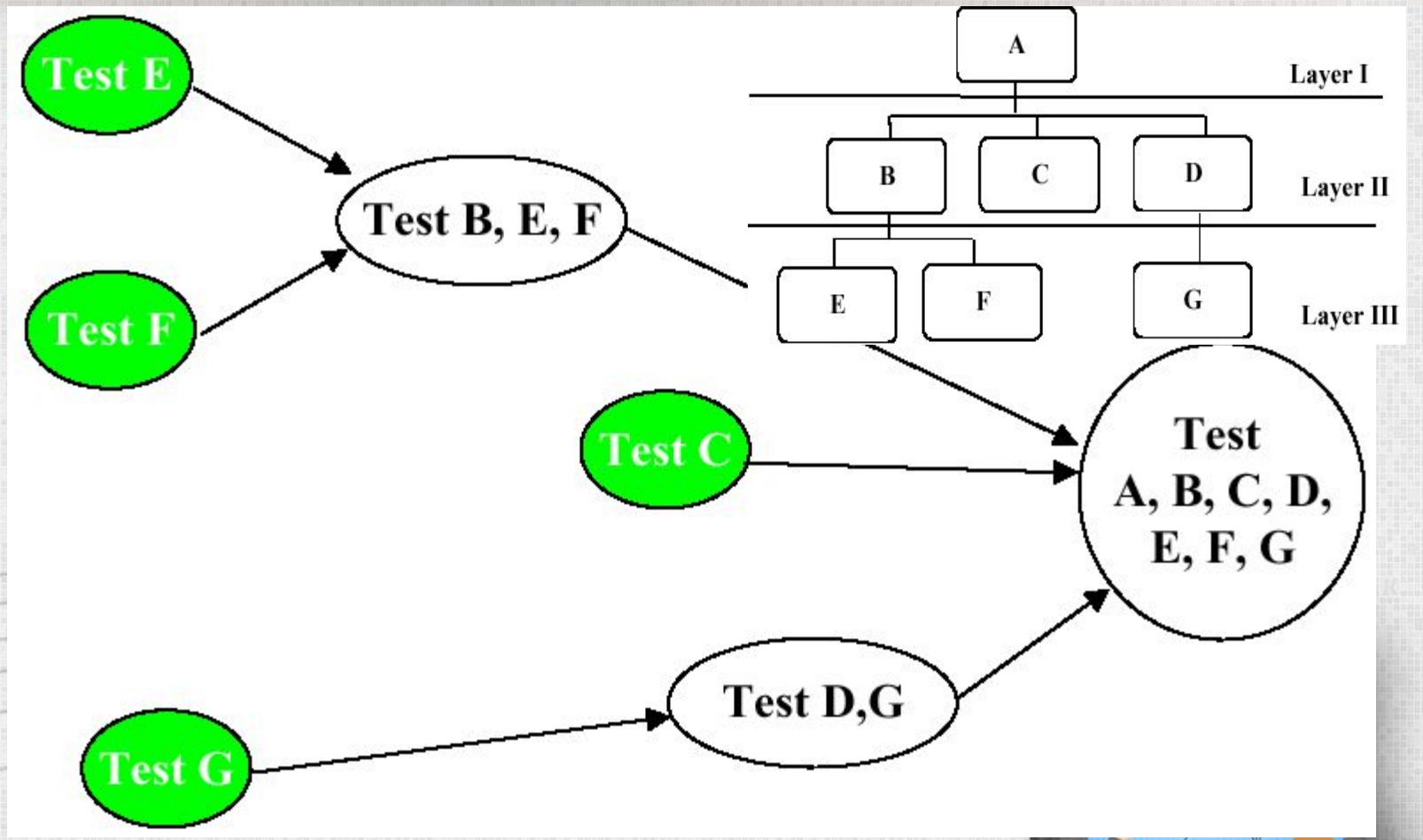
Bottom-up Integration Testing (2)

- In this approach, lower level modules are tested extensively thus make sure that highest used module is tested properly.





Bottom-up Testing Graphical Representation

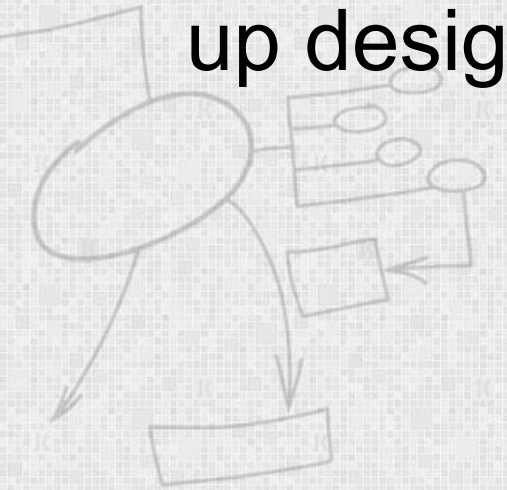


Comments on Graphical Representation

- Modules E and F are tested. Then modules B, E, F are tested.
- Module F is tested. Then modules F and G are tested.
- Module C is tested.
- Finally – modules A, B, C, D, E, F, G are tested.

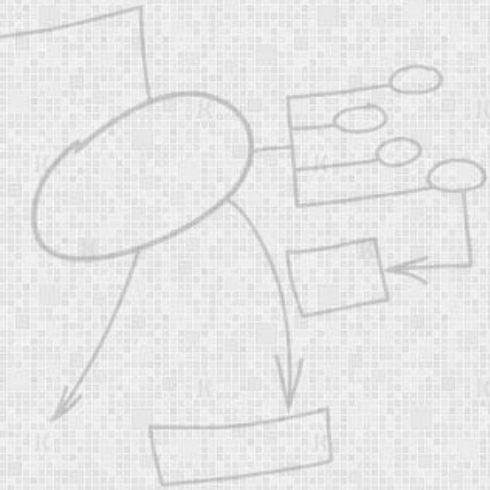
Advantages of Bottom-up Testing

- Behavior of the interaction points are crystal clear, as components are added in the controlled manner and tested repetitively.
- Appropriate for applications where bottom up design methodology is used.



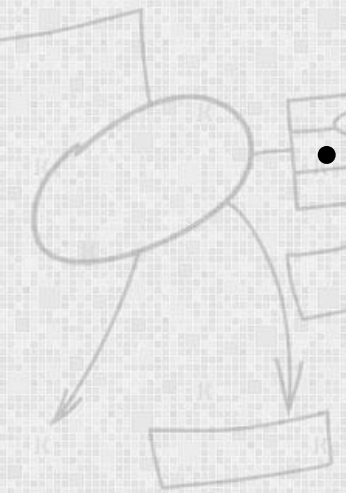
Disadvantages of Bottom-up Testing

- Writing and maintaining test drivers is more difficult than writing stubs.



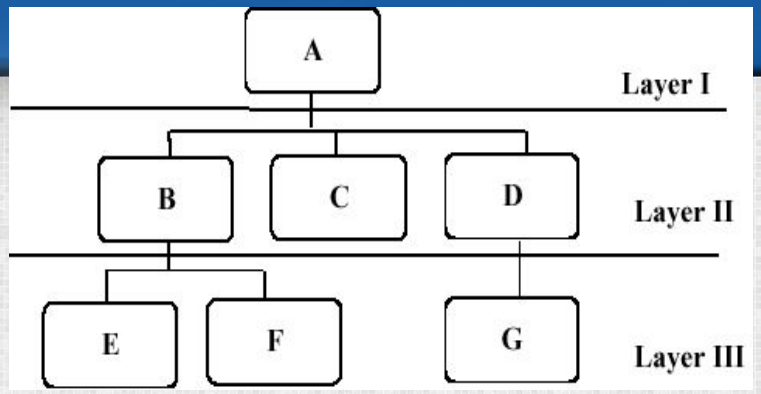
Top-down Testing

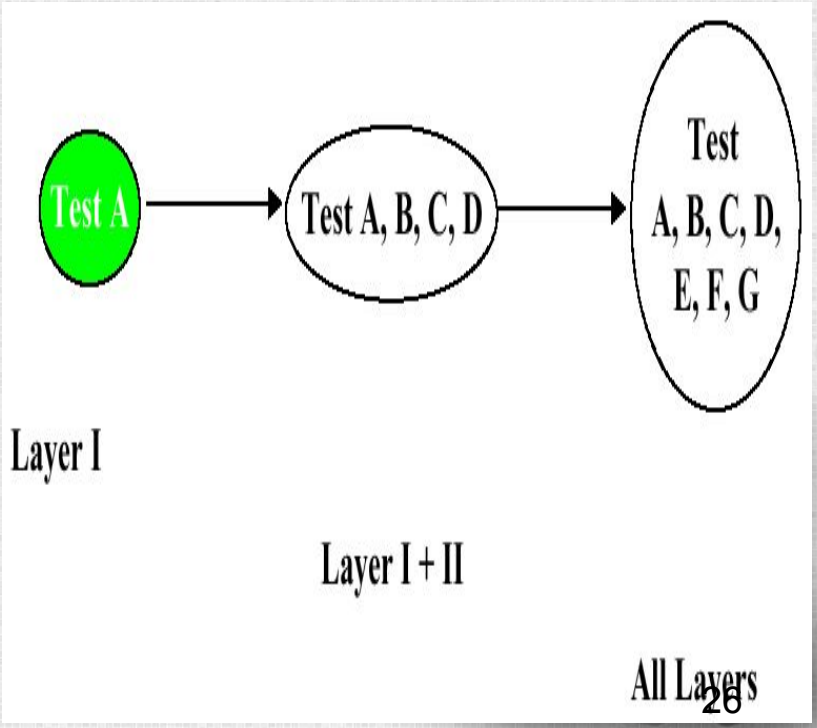
- Top down integration testing is an incremental integration testing technique which begins by testing the top level module and progressively adds in lower level module one by one.
- Lower level modules are normally simulated by stubs which mimic functionality of lower level modules.
- As you add lower level code, you will replace stubs with the actual components.



Advantages of Top-down Testing

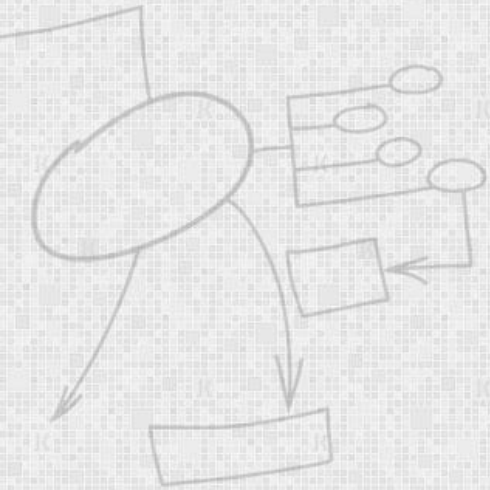
- Driver do not have to be written when top down testing is used.
- It provides early working module of the program and so design defects can be found and corrected early.





Disadvantages of Top-down Testing

- Stubs have to be written with utmost care as they will simulate setting of output parameters.



Mixed Integration Testing

- A mixed integration testing follows a combination of top-down and bottom-up testing approaches.

- In top-down approach, testing can start only after the top-level modules have been coded and unit tested.

Mixed Integration Testing

- Similarly, bottom-up testing can start only after the bottom level modules are ready.
- The mixed approach overcomes this shortcoming of the top-down and bottom-up approaches.
- In the mixed testing approaches, testing can start as and when modules become available. Therefore, this is one of the most commonly used integration testing approaches.